Пользовательская инструкция

AI Платформа AIналитик

Скачать: https://github.com/chaussky/ainalyst.git

Глава 5 — Requirements Life Cycle Management

Общая характеристика Главы 5

Глава 5 BABOK — «Requirements Life Cycle Management» — это то, что происходит с требованиями **после** их выявления. Если Глава 4 производит сырьё (транскрипты, факты, наблюдения), то Глава 5 превращает это сырьё в управляемый, живой реестр требований, который сопровождает проект от старта до сдачи.

Ключевая идея главы: требования — не документ, написанный один раз. Они живут, меняются, стареют, конфликтуют, получают статусы, обрастают связями. Задача BA — быть «смотрителем» этого реестра на протяжении всего проекта.

Глава 5 отвечает на пять главных вопросов практикующего BA:

- **5.1** — *Откуда взялось это требование, и что изменится, если оно поменяется?* (трассировка)
- **5.2** — *Актуальны ли мои требования прямо сейчас?* (поддержание)
- **5.3** — *Что из этого делать сначала, и как согласовать мнения разных стейкхолдеров?* (приоритизация)
- **5.4** — *Стоит ли принимать это изменение, и что за ним тянется?* (оценка изменений)
- **5.5** — *Требования согласованы официально?* (утверждение и baseline)

Работа по Главе 5 идёт параллельно с Главой 4: каждое подтверждённое требование из 4.3 сразу попадает в инфраструктуру Главы 5. А потом задачи 5.1–5.5 сопровождают проект до самого перехода в Главу 6.

Задача 5.1 — Trace Requirements (Трассировка требований)

Краткое описание

BA строит и поддерживает граф связей между всеми артефактами проекта: бизнес-потребностями, требованиями стейкхолдеров, функциональными требованиями, тестами, компонентами. Этот граф — «скелет», который держит проект вместе и делает любое изменение управляемым.

Боли и проблемы BA

Изменение требования — это непредсказуемая реакция. Пришёл CR: «поменяем логику расчёта скидки». BA меняет FR-014. И только потом, в середине спринта, выясняется: FR-014 зависит от FR-022, FR-022 связано с NFR-005 по производительности, и к этой же цепочке привязано 4 тест-кейса в QA. Ничего этого не было видно заранее, потому что связи держались в голове или в Excel.

«Откуда взялось это требование?» — типичный вопрос при ревью или аудите. BA листает ноутбук, пытается вспомнить. В регуляторных проектах (банки, фарма, госсектор) отсутствие трассировки — это уже не просто риск, а нарушение требований compliance.

Покрытие проверяется вручную или не проверяется вовсе. Перед передачей в разработку BA не знает: у всех ли требований есть «родитель» (бизнес-потребность)? Все ли требования покрыты тест-кейсами? Все ли функции реализованы? Такой аудит вручную — это часы работы в Excel.

«Мёртвая» трассировка. BA создал матрицу в начале проекта — красиво. Потом требования менялись, а матрица нет. Через три месяца она описывает реальность только наполовину. Стейкхолдер, который на неё опирается, работает с устаревшими данными.

Что мы реализовали

Граф связей как живой репозиторий. `init_traceability_repo` создаёт структурированный JSON-репозиторий проекта. С этого момента каждое требование — это узел графа, а каждая связь между ними — рёбра с типом: `derives` (происходит из), `depends` (зависит от), `satisfies` (реализует), `verifies` (проверяет), `modifies` (изменяет, для CR).

Добавление связей в один шаг. `add_trace_link` принимает два узла и тип связи. BA не заполняет таблицы — он описывает ситуацию голосом: *«FR-014 происходит из SR-003, запиши»*, AIналитик вызывает инструмент. Граф растёт автоматически.

Impact analysis за секунды. `run_impact_analysis` обходит граф в глубину от изменённого требования и возвращает полный список затронутых артефактов: какие требования, тесты, компоненты, CR-ы. Это механика для задачи 5.4 — оценки стоимости изменения.

Аудит покрытия. `check_coverage` проверяет весь граф и возвращает список проблем с цветовой маркировкой: 🔴 требование без источника (orphan), ☐ требование без реализации или без теста, ☐ полное покрытие. Это делается перед приоритизацией и перед утверждением.

Матрица трассировки по требованию. `export_traceability_matrix` генерирует готовую матрицу в Markdown. Её можно сразу включить в пакет согласования для стейкхолдеров или отправить на аудит.

Три пресета формальности. Lite (Agile, стартап), Standard (средние проекты), Full (regulated, внешний аудит, enterprise). Выбирается один раз под контекст проекта. BA не гадает, какой уровень документации нужен — скилл рекомендует на основе нескольких вопросов.

Ценности для BA

Любое изменение — управляемое. Вместо «давайте посмотрим что затронет» — моментальный список всего затронутого с указанием типа связи. BA может принести на встречу с заказчиком полный impact report, сгенерированный за 10 секунд, а не за несколько часов.

Готовность к аудиту в любой момент. Трассировка хранится в структурированном JSON, из которого в один шаг генерируется матрица. Для регуляторных проектов это принципиально: *«покажите трассировку требования FR-047 к бизнес-потребности»* — ответ готов немедленно.

Покрытие не упускается. `check_coverage` перед каждым ключевым шагом (приоритизация, утверждение) гарантирует что BA не передаёт в следующий этап требования без бизнес-обоснования. Это снижает риск споров на этапе разработки: «зачем это вообще делать?»

Трассировка живёт вместе с проектом. Каждый CR, каждое изменение требования — проходит через граф. Матрица не устаревает, потому что обновляется не вручную, а через инструменты скилла.

Как пользоваться: пример

Проект стартовал, из задачи 4.3 получены первые подтверждённые требования. BA говорит:

«Начинаем трассировку по проекту CRM. У нас Agile, команда 12 человек, внешнего аудита нет.»

AIналитик рекомендует пресет Standard, инициализирует репозиторий. Дальше BA добавляет первое требование:

«FR-001 — Авторизация пользователя. Происходит из SR-002 — требование безопасности от IT-директора.»

AIналитик вызывает `add_trace_link`, фиксирует связь `FR-001 derives SR-002`. После нескольких итераций BA просит проверку:

«Сделай аудит покрытия перед тем как мы уйдём на приоритизацию.»

`check_coverage` возвращает: FR-007 без родителя (🔴), NFR-003 без теста (☐), остальные в порядке (☐). BA выясняет у аналитиков откуда взялся FR-007 — оказывается, это требование

вообще не было подтверждено в 4.3, добавили «на ходу». Проблема выявлена до, а не во время разработки.

Задача 5.2 — Maintain Requirements (Поддержание требований)

Краткое описание

BA следит за актуальностью реестра требований: обновляет статусы и версии, помечает устаревшие требования, регулярно проводит аудит «здоровья» реестра, выявляет кандидатов на повторное использование в других инициативах. Задача сквозная — она выполняется на протяжении всего проекта.

Боли и проблемы BA

Реестр застывает в момент своего создания. BA собрал требования, зафиксировал — и больше не возвращается. Через месяц активной работы: 5 требований изменились содержательно, 3 потеряли актуальность, у 2 сменился владелец. В реестре всё по-прежнему как в день создания.

Версионность требований не ведётся. Пришёл CR, BA поправил формулировку FR-017. Прошлая версия не сохранена. Через неделю заказчик: *«почему в требовании теперь написано вот это? Мы же договорились иначе»*. BA не может показать историю изменений — нет документального следа.

Статусы требований не актуальны. Требование в статусе «на согласовании» уже три недели — но это потому что согласование прошло, просто никто не обновил. Разработчик смотрит в реестр и не понимает: брать в работу или ждать?

«Дрейф Must». К концу этапа выявления в категории Must оказывается 70% требований. Все «обязательные», приоритет у всего высокий. Реестр теряет структуру — он больше не помогает принимать решения, он только создаёт вид работы.

Повторное использование не происходит. BA начинает новый проект и пишет требования с нуля. Хотя в предыдущем проекте уже было «Пользователь должен авторизоваться через корпоративный SSO» — требование, применимое в трёх из пяти инициатив. Но оно лежит в папке прошлого проекта и о нём никто не вспоминает.

Что мы реализовали

Обновление требований с историей. `update_requirement` обновляет атрибуты требования — статус, приоритет, формулировку, версию — и автоматически сохраняет историю: что было, что стало, дата, причина. Через три месяца можно посмотреть полную хронологию любого требования.

Правильное устаревание. `deprecate_requirements` помечает требование как устаревшее с выбором финального статуса: `deprecated` (устарело, замены нет), `superseded` (заменено другим требованием, указывается чем), `retired` (проект завершён). Требование не удаляется из репозитория — история сохраняется для аудита.

Аудит здоровья реестра. `check_requirements_health` анализирует весь реестр и возвращает: 🔴 высоковолатильные требования (менялись слишком часто — это сигнал нестабильного стейкхолдера или плохой формулировки), ☐ требования которые давно не обновлялись (возможно устарели), ☐ требования надолго застрявшие в статусе `draft`. По каждой позиции — рекомендация действия.

Поиск кандидатов на повторное использование. `find_reusable_requirements` фильтрует реестр по типу, теме, статусу и флагу `reuse_candidate`. Возвращает список требований с оценкой пригодности: стабильные, сформулированные без привязки к конкретной системе, с финальным статусом — лучшие кандидаты.

Автоматическая детекция «дрейфа Must». При аудите `check_requirements_health` система считает долю Must в общем количестве требований. Если она превышает порог (по умолчанию 40%) — BA получает предупреждение. Это сигнал для пересмотра перед следующей сессией приоритизации.

Ценности для BA

Реестр отражает реальность, а не историю. Когда разработчик открывает реестр перед спринтом, он видит актуальные статусы, текущие версии, актуальных владельцев. Время на вопросы «а это уже согласовано?» и «а это ещё актуально?» сокращается в разы.

История изменений — это доказательная база. При любом «мы же договорились по-другому» BA открывает историю конкретного требования: версия 1.0 выглядела так, 23 февраля пришёл CR, версия 1.1 согласована с такими-то. Разговор переходит в конструктивное русло немедленно.

Здоровый реестр к каждому ключевому шагу. `check_requirements_health` запускается перед приоритизацией (5.3) и перед утверждением (5.5). Это гарантирует что BA уходит на следующий этап с чистым, актуальным материалом, а не с накопленным техническим долгом.

Экономия на старте новых проектов. `find_reusable_requirements` — это поиск по всем прошлым проектам в платформе. Готовое требование из прошлого проекта с историей изменений и статусом `approved` экономит часы работы по выявлению и формулировке. Особенно ценно в организациях с множеством похожих инициатив.

Как пользоваться: пример

Середина проекта. Пришёл CR: заказчик хочет изменить логику уведомлений. BA попросил разработчиков внести правки в FR-031, 5.4 уже одобрила CR. Теперь нужно обновить реестр.

«Обнови FR-031: статус меняется на `under_change`, версия с 1.0 на 1.1, формулировка вот такая. Причина — CR-005, одобрен спонсором.»

AIналитик обновляет, история фиксируется. Через месяц аудит:

«Сделай проверку здоровья реестра перед тем как мы уйдём на согласование.»

`check_requirements_health` возвращает: NFR-002 — последнее обновление 6 недель назад, возможно устарело; FR-019 — в статусе `draft` уже 3 недели. BA уточняет у стейкхолдеров, закрывает вопросы. На согласование уходит чистый реестр.

Задача 5.3 — Prioritize Requirements (Приоритизация требований)

Краткое описание

BA определяет относительную важность требований для стейкхолдеров, выбирает метод приоритизации под контекст проекта, собирает оценки, агрегирует их, выявляет конфликты между стейкхолдерами и фасилитирует их разрешение. Задача повторяется при каждом цикле планирования — не только один раз.

Боли и проблемы BA

«Всё важно». Классическая ситуация: после выявления 80 требований и 5 стейкхолдеров, каждый из которых считает свои требования приоритетными, BA должен составить осмысленный порядок реализации. Без метода — это невозможно. С методом — нужно ещё убедить стейкхолдеров.

Конфликты между стейкхолдерами не документируются. Директор по продажам хочет FR-024 в первый спринт. IT-директор — FR-048. Оба правы с точки зрения своего подразделения. BA фиксирует конфликт у себя в голове и пытается разрулить на встрече. Если встречи не было — конфликт остаётся неразрешённым и всплывает в разработке.

Выбор метода приоритизации — интуитивный. Большинство BA используют MoSCoW потому что все так делают. MoSCoW хорош для категоризации, но плохо работает при большом количестве Must и не учитывает стоимость реализации. WSJF лучше для Agile, но требует оценок от разработчиков, которых у BA может не быть. Правильный метод — тот, который подходит под контекст.

Dependency violations не проверяются. FR-019 поставили в `Could`, FR-020 — в `Must`. Но FR-020 зависит от FR-019 — без него его невозможно реализовать. Конфликт обнаруживается в планировании спринта, не в приоритизации.

Нестабильные требования получают высокий приоритет. FR-011 поставили в Must, хотя его формулировка менялась 4 раза за 3 недели. Планировать Must-требование с высокой волатильностью — риск для командного ресурса: сделают, а оно снова поменяется.

Что мы реализовали

Три метода приоритизации под контекст. Скилл поддерживает MoSCoW (быстро, понятно стейкхолдерам), WSJF (числовое ранжирование с учётом стоимости и time-criticality, лучше для Agile), Impact/Effort Matrix (визуальный, удобен для воркшопов). Выбор метода не интуитивный — скилл задаёт вопросы о контексте и рекомендует.

Сессионный сбор оценок. `start_prioritization_session` открывает сессию и формирует список требований для оценки. `add_stakeholder_scores` собирает оценки по одному стейкхолдеру за раз. Это позволяет проводить оценку асинхронно: каждый стейкхолдер даёт оценки в удобное время, BA агрегирует всё одной командой.

Автоматическая агрегация и выявление конфликтов. `run_aggregation` агрегирует оценки с учётом influence-весов стейкхолдеров (из реестра 4.2) и автоматически выявляет: dependency violations (приоритет выше зависимости — невозможно), конфликты между стейкхолдерами (большой разброс оценок), волатильные требования с высоким приоритетом. Каждый конфликт — отдельная запись с типом и участниками.

Структурированное разрешение конфликтов. `resolve_conflict` фиксирует разрешение каждого конфликта: метод (эскалация, компромисс, экспертная оценка, pilot-тест), участников, итоговое решение, обоснование. Это не просто «поговорили» — это документальная запись, защищающая BA.

Итог → в репозиторий. `save_prioritization_result` обновляет атрибуты приоритета в репозитории 5.2 для каждого требования и сохраняет Markdown-отчёт по сессии: метод, участники, итоговые приоритеты, разрешённые конфликты, обоснование. Это документ для стейкхолдеров.

Ценности для BA

Приоритизация становится защищаемой, а не интуитивной. Когда заказчик через месяц спрашивает «почему FR-019 не вошло в первый релиз?» — BA открывает отчёт сессии приоритизации: вот оценки стейкхолдеров, вот агрегация с весами, вот решение. Это разговор с данными, а не с воспоминаниями.

Конфликты видны до разработки. Dependency violation между FR-019 и FR-020, выявленный при агрегации — это несколько минут работы BA по устранению. Тот же конфликт, обнаруженный в середине спринта — это пересмотр планирования и объяснения команде.

Стейкхолдеры вовлечены асинхронно. BA не собирает всех одновременно на двухчасовой воркшоп (и не борется с расписаниями директоров). Оценки собираются отдельно по каждому, агрегация — после. Для большого количества стейкхолдеров это принципиальная экономия.

Нестабильные требования не попадают в Must без предупреждения. Флаг волатильности в `run_aggregation` не запрещает BA ставить нестабильное требование в высокий приоритет — но заставляет сделать это осознанно, с пониманием риска.

Как пользоваться: пример

Завершился этап выявления, BA готов к первой сессии приоритизации. 45 требований, 4 стейкхолдера с разными influence-весами.

«Открой сессию приоритизации по проекту CRM. Метод MoSCoW, будем оценивать все 45 подтверждённых требований.»

AIналитик открывает сессию, формирует список. BA проводит оценки с каждым стейкхолдером отдельно и вносит их через `add_stakeholder_scores`. После четвёртого стейкхолдера:

«Агрегируй результаты.»

`run_aggregation` возвращает: 3 конфликта (FR-007 — большой разброс между продажами и IT), 1 dependency violation (FR-031 в Must, но зависит от FR-044 в Could), 2 волатильных требования в Must. BA разрешает конфликты, корректирует очевидный dependency violation, сохраняет результат.
«Сохрани итоги сессии.» — готов отчёт для стейкхолдеров.

Задача 5.4 — Assess Requirements Changes (Оценка изменений требований)

Краткое описание

BA играет роль «привратника изменений»: структурировано оценивает каждый Change Request, считает его влияние на граф требований, скорит по формуле бизнес- и технических осей, готовит обоснованную рекомендацию для спонсора или CCB. Решение принимает не BA — он готовит аргументированный материал для принятия решения.

Боли и проблемы BA

Изменения принимаются под давлением, а не по анализу. Приходит менеджер проекта: «заказчик хочет добавить эту фичу, давай быстренько». BA говорит ок. Через неделю: оказывается, эта фича конфликтует с двумя существующими требованиями, тянет за собой переделку архитектуры, и заказчик не осознавал эту цену, когда просил.

«Стоимость» изменения никто не считает системно. Impact analysis делается вручную (если делается вообще). BA обходит по памяти связанные требования — и неизбежно что-то упускает. Итог: CR принят, а затронутые артефакты выясняются уже в разработке.

Каждый CR — это «а не связано ли это с тем, что мы уже решили?» Изменение FR-017 может конфликтовать с decision из протокола встречи трёхнедельной давности. BA не помнит все принятые решения по проекту, а лезть в архив некогда.

Нет единого места для истории всех CR. Один CR — в письме, другой — в Jira, третий — вообще устно договорились. Через месяц невозможно восстановить: сколько CR было принято? Какие требования сейчас находятся под изменением? Что из обещанного ещё не реализовано?

Решение «брать / не брать» без структуры. BA даёт рекомендацию интуитивно или под давлением. Без скоринга по бизнес-осям и техническому влиянию — это не рекомендация, это мнение. Спонсор принимает решение без данных.

Что мы реализовали

Стадийный pipeline для каждого CR. `open_cr` → `run_cr_impact` → `score_cr` → `resolve_cr`. Это не лишняя формальность — каждый шаг добавляет данные, которые нужны на следующем. Для небольшого CR цикл проходится быстро; для крупного — даёт полную картину.

CR как узел в графе трассировки. Каждый Change Request хранится в репозитории 5.1 как узел с типом `change_request`, связанный с затронутыми требованиями связью `modifies`. Вся история изменений видна прямо в матрице трассировки — не нужно отдельного реестра CR.

Автоматический impact analysis. `run_cr_impact` использует граф трассировки из 5.1 (BFS-обход) и возвращает полный список затронутых артефактов: требования, тест-кейсы, компоненты. Это технический input для скоринга — BA не пропустит ничего что «тянется» за изменением.

Скоринг по формуле с обоснованием. `score_cr` считает: `Score = Benefit×2 + Urgency×1.5 + Impact×1 - Cost×1.5 - ScheduleRisk×1`. Технические оси (Impact, ScheduleRisk) вычисляются автоматически из BFS-обхода. Бизнес-оси (Benefit, Cost, Urgency) вводит BA после консультации с заказчиком. Формула даёт предварительный вердикт (Approve / Modify / Defer / Reject), AIналитик добавляет текстовое обоснование.

Автообновление статусов при решении. `resolve_cr` при решении Approved автоматически переводит затронутые требования в статус `under_change`. BA не забывает обновить реестр вручную. CR Decision Record сохраняется и уходит в 4.4 (коммуникация стейкхолдерам) и в 5.5 (как контекст для утверждения).

Ценности для BA

Рекомендация с данными, а не с мнением. BA приходит к спонсору не с «я думаю, стоит принять», а с: технически затронуто 7 артефактов, скор 6.5 (Modify), вот что нужно изменить чтобы снизить риск. Решение принимается с пониманием последствий — и ответственность правильно распределена.

История всех CR в одном месте. `open_cr` с уникальным ID, связи в графе трассировки, Decision Records — это полный аудиторский след. Через 3 месяца: сколько CR было? Какие приняты, какие отклонены? Почему? Ответ моментальный.

Изменения не «улетают в разработку» без оценки. Pipeline с обязательным impact analysis создаёт барьер: прежде чем рекомендовать принять CR, BA понимает что за ним тянется. Это снижает количество сюрпризов в разработке — одну из главных причин задержек.

BA защищён при scope creep. Когда менеджер через два месяца удивляется что «мы добавили так много» — BA открывает реестр CR: вот 12 принятых изменений, вот даты, вот Decision Records с подписью спонсора. Это профессиональная документация scope creep, а не апелляция к памяти.

Как пользоваться: пример

Поступает запрос: заказчик хочет добавить интеграцию с новой CRM-системой, о которой не было речи в начале.

«Открой Change Request: заказчик просит добавить интеграцию с Salesforce. Влияет на FR-022, FR-033, скорее всего ещё на что-то.»

`open_cr` создаёт CR-008. `run_cr_impact` пробегает по графу трассировки — оказывается, затронуты ещё NFR-004 (производительность API) и 3 тест-кейса. BA фиксирует оценки бизнес-осей после разговора с заказчиком, запускает скоринг. Скор 4.2 — «Modify»: принять, но с изменёнными условиями по срокам. BA готовит рекомендацию для спонсора, тот принимает решение, `resolve_cr` обновляет все статусы.

Задача 5.5 — Approve Requirements (Утверждение требований)

Краткое описание

BA организует официальное согласование требований со стейкхолдерами и создаёт Requirements Baseline — зафиксированную версию требований, которая становится контрактом для разработки. Задача поддерживает как Predictive-подход (baseline в конце фазы), так и Agile (Sprint Backlog Baseline).

Боли и проблемы BA

«Согласовано» означает разное для разных людей. Кто-то кивнул на встрече, кто-то написал «ок» в письме, кто-то вообще не ответил — BA всё равно считает это согласованием. Потом приходит разработчик: «а мне никто не говорил что FR-019 утверждено, я ждал официального подтверждения». Или стейкхолдер: «я же не подписывал ничего, как вы могли это брать в работу».

Условные согласования теряются. Стейкхолдер дал согласие с условием: «принимаю, но только если добавите раздел о безопасности». BA зафиксировал у себя, пообещал. Условие выполнено через неделю — но нигде не закрыто официально. Стейкхолдер через месяц: «а вы добавили то, о чём мы договорились?»

Нет понимания готовности пакета к baseline. BA собрал ответы от 5 стейкхолдеров. Три одобрили, один — с условием, один — отклонил. Можно ли уже создавать baseline? Кто из них Accountable, а кто Consulted? Блокирует ли отклонение от Consulted?

Разные форматы пакета для разных аудиторий. Бизнес-заказчику нужно одно, разработчикам — другое, регулятору — третье. BA готовит один документ «для всех» — и ни одна из аудиторий не получает то, что ей действительно нужно.

Baseline нет как такового. Требования «согласованы» где-то в переписке, но нет зафиксированной версии, нет даты, нет списка кто согласовал. Через месяц невозможно ответить: «что именно было в baseline v1.0?»

Что мы реализовали

Пакет согласования с адаптацией под аудиторию. `prepare_approval_package` собирает требования, добавляет матрицу трассировки (5.1), приоритеты (5.3) и CR Decision Records (5.4), и форматирует документ под конкретную аудиторию: `business` — без технических деталей, `developer` — с техническими атрибутами и зависимостями, `regulator` — с полной трассировкой и историей изменений.

Фиксация каждого решения стейкхолдера. `record_approval_decision` записывает ответ каждого участника: Approved, Conditional (с текстом условия, дедлайном, ответственным), Rejected (с причиной), Abstained. Можно фиксировать решения как по всему пакету, так и по отдельным требованиям внутри него.

Закрытие условных одобрений. `close_approval_condition` фиксирует выполнение условия: что было сделано, дата закрытия. Требование переходит из `conditional_approved` в `approved`. История условия сохраняется.

Готовность к baseline — по критериям, а не по ощущению. `check_approval_status` считает: сколько Approved / Conditional / Rejected / ещё не ответили. Выявляет просроченные условия. Флагует rejected от Accountable-стейкхолдеров (это блокеры). Даёт чёткий вердикт: готово к baseline или нет, и почему.

Официальный Requirements Baseline. `create_requirements_baseline` создаёт snapshot пакета в `{project}_approval_history.json`, обновляет статусы требований в репозитории 5.1 (→ `approved`), генерирует Approval Record — Markdown-документ с версией, датой, списком согласовавших. Этот артефакт уходит в 4.4 (коммуникация) и становится входом для Главы 6.

Поддержка Predictive и Agile. В Predictive-проекте baseline создаётся в конце фазы — включает все требования этапа. В Agile — перед каждым спринтом Product Owner согласует Sprint Backlog Baseline с конкретным номером спринта.

Ценности для BA

«Согласовано» теперь имеет точный смысл. Есть Approval Record с датой, версией baseline, списком участников и их решениями. Нет больше ситуации «ну мы же обсуждали на встрече». Есть документальный след, который защищает BA и даёт разработчикам уверенность в материале.

Conditional-согласования не теряются. Каждое условие фиксируется с дедлайном и ответственным. `check_approval_status` покажет просроченные условия — BA не пропустит их в потоке других задач. Baseline не будет создан пока условие не закрыто (если только BA не

использует `force`-флаг осознанно).

BA знает когда можно создавать baseline. `check_approval_status` снимает неопределённость: вот 4 Approved, вот 1 Conditional с открытым условием, вот 1 Rejected от Consulted (не блокирует). Вердикт: можно создавать baseline с документированным риском по отклонению Consulted. Или: нет, Accountable не одобрил, нужно разрешить конфликт.

Разные стейкхолдеры получают нужный им формат. Директор по продажам читает бизнес-версию пакета без технического языка. Разработчики — версию с зависимостями и атрибутами. Это снижает количество вопросов «что это значит?» и ускоряет цикл согласования.

Baseline — это юридически значимый момент проекта. После создания baseline изменить требования можно только через официальный CR (5.4). Это не ограничение — это защита BA от неформального scope creep. «Заказчик хочет добавить...» — хорошо, откроем CR.

Как пользоваться: пример

Завершена фаза выявления, все требования прошли 4.3 и занесены в 5.1. Первая сессия согласования.

«Подготовь пакет согласования по проекту CRM. Predictive-подход, требования FR-001 до FR-045. Для бизнес-аудитории — нам идёт к директору по продажам и финдиректору.»

`prepare_approval_package` собирает пакет APKG-001. BA отправляет документ через 4.4. Приходят ответы:
«Директор по продажам — одобрил всё. Финдиректор — принял, но с условием: FR-037 (интеграция с 1С) должна пройти security review до начала разработки. Технический директор — воздержался.»

BA фиксирует каждое решение через `record_approval_decision`. `check_approval_status` возвращает: 1 Approved, 1 Conditional (дедлайн через 2 недели), 1 Abstained. Вердикт: не готов к baseline, нужно закрыть условие финдиректора.

Через 10 дней security review пройден. *«Закрой условие финдиректора по FR-037: security review завершён, протокол прикреплён.»* `close_approval_condition` фиксирует выполнение.

«Создай baseline v1.0.» — Approval Record сохранён, требования получили статус `approved`, проект переходит в Главу 6.

Финальный синтез по Главе 5

Глава 5 — это инфраструктура качества всего проекта. Если Глава 4 производит сырьё, то Глава 5 даёт этому сырью структуру, историю, статус и официальный вес. Всё что будет делаться в Главах 6, 7 и 8 — строится на фундаменте, заложенном в Главе 5.

Каждая задача Главы 5 снимает конкретный класс риска:

- 5.1 убирает риск непредсказуемых последствий изменений

- 5.2 убирает риск работы с устаревшими данными
- 5.3 убирает риск субъективной приоритизации и скрытых конфликтов
- 5.4 убирает риск неуправляемого scope creep
- 5.5 убирает риск спорного согласования и размытой ответственности

Ответственность BA в Главе 5 сосредоточена там, где она и должна быть: аналитические решения (какой уровень формальности трассировки нужен?), фасилитация (как разрешить конфликт приоритетов?), оценка (какова бизнес-ценность этого CR?). Платформа берёт на себя инфраструктуру: хранение, версионирование, агрегацию, генерацию отчётов, проверку покрытия.

Практический результат: BA, работающий через Главу 5 платформы, к моменту перехода в Главу 6 приходит с чётким ответом на вопрос «что именно мы разрабатываем?» — с утверждённым baseline, полной трассировкой, актуальным реестром и историей всех изменений. Это не бюрократия ради бюрократии — это условие для качественной работы на последующих этапах.